P6 - Open Braille Translator

Multi-purpose braille display firmware for Unified English Braille

Authors
Johan Nissen Riedel

AALBORG UNIVERSITET

Aalborg University
Electronics and IT

Department of Computer Science
Aalborg University
http://www.aau.dk

Titel:
Open Braille Translator

Theme:
Multi-purpose braille display firmware for Unified English Braille

Project periode:
Sixth semester

Project group:
SW6 Group 12

Group member:
Johan Nissen Riedel

Supervisor:
Tiantian Liu

Number of pages: 34

Due date:
June 24, 2026

Abstract:

With lack of access to sufficient education in braille, many visually impaired people are at risk of becoming illiterate. Studies suggest that there is a strong link between braille literacy and career success, leading to an overall higher quality of life for blind individuals. For blind people with braille literacy, assistive technologies, such as refreshable braille displays and screen readers provide a convenient, but expensive way to interface with computers, to the benefit of their personal independence. Recent initiatives to improve access to braille education coupled with major developments in braille display technology have the potential to reverse the negative trend in braille literacy and increase the adoption rate of refreshable braille displays. This report will describe the design and implementation of a software solution that attempts to bridge the gap between braille displays and computers, resulting in a more refined experience for the visually impaired, particularly in programming and academic contexts. With Unified English Braille as the braille system of choice, the solution makes heavy use of its wide variety of characters to convey extracted text as closely as possible. The final implementation includes a graphical user interface to simulate a braille display with buttons for switching pages, two custom translation tools that can convert text to contracted and uncontracted Unified English Braille, two external plugins for extracting text from a code editor and images in a web browser, and a basic text-to-speech system.

Summary

This report describes the process of design and implementation of a software solution intended to aid visually impaired people in interfacing with computers through refreshable braille displays. The initial chapter will describe the consequences of lacking braille literacy, which studies claim are severe, resulting in higher unemployment rates and less academic success. This is followed by a brief description of braille, going over its advent and evolution through time, that has made it branch into many different systems. As much of the report details the complex rules of Unified English Braille, relevant braille terms and their explanations are included in the introductory chapter. In the problem delineation chapter, additional studies and numbers regarding the importance of braille literacy are outlined, further emphasizing its importance. A number of reasons are listed as contributors to falling braille literacy rates, including a lack of qualified braille teachers and poor enforcement of legislation, since blind students are required by federal law to receive education. Additionally, there is no general consensus on what it means to be a qualified braille instructor. One report describes braille literacy as the key to success and claims that 90% of blind people who are employed can read and write braille. Next, some common assistive technologies are listed and explained, such as the screen reader, which allows a blind user "see" elements on the computer screen by making use of accessibility APIs and various text extraction tools. Of particular note is the Refreshable Braille Display, which allows users to read text and other elements converted to braille on a physical display with one or more rows of braille cells. These physical displays are very useful, but unfortunately expensive and thus, not a viable product for many individuals. However, a later section describes recent developments in braille display technology, which are already drastically cutting the prices of displays, and have the potential of making them affordable to most people in the near future. The previously describe points are used to produce the following problem statement: *How can I design and implement a software solution to aid the visually impaired in using computers for multiple contexts with a primary focus on supporting a wide variety of Refreshable Braille Display models?* The design chapter is introduced with a brief description of the desired product: a central application, able to receive data from a variety of sources using Inter-Process Communication and the extensive Unified English Braille system. The choice of braille system is explained in the next section, where different braille systems are listed and described, such as Grade 1 English braille. Their benefits and downsides are discussed, until the choice to use Unified English Braille is made. The development chapter describes the process of representing braille cells in a digital format by using Python to define different classes and methods. These digital representations are used to in the translation process, where the development of two different Context-Free Grammars representing contracted and uncontracted Unified English Braille (UEB) is described. A brief explanation of parsing and traversing parse trees is included in this chapter. The many rules of contracted UEB are also described in this chapter while they are implemented in the translator. Next, the process of simulating the braille display output with a graphical user interface is described. This section highlights how the software is easy to modify for braille displays of any number of rows and columns. Next, the implementation of IPC is described, which uses TCP sockets and web sockets to receive data from a variety of external plugins, such as a Visual Studio Code plugin and a Firefox plugin. Finally in the development chapter, our implementation of a basic text-to-speech system is described. This chapter is followed by a lengthy discussion of potential future improvements that they solution can receive. It highlights a number of shortcomings in the development process, primarily stemming the tool used to help with the translator. Finally, the conclusion chapter talks about the fulfilment of the problem statement, which is described as a partial fulfilment.

Contents

1 Introduction

In their daily lives, people with visual impairments are expected by society to overcome a number of obstacles that pose no additional challenge to sighted individuals. For people in academic or professional contexts, these challenges can include reading e-mails, textbooks, lecture slides and other types of documents. Many people with visual impairments will opt to use software that reads the text aloud, that is, text-to-speech programs, while a smaller subset uses physical Braille displays, which can allow for increased reading speeds.[1] Unfortunately, the high cost, coupled with falling braille literacy rates, makes physical displays an unviable option for many people. However, recent developments in Braille display technology that point towards a significant decrease in manufacturing costs, combined with efforts to increase braille literacy could reverse the trend of decreasing literacy rates and thus increase the productivity, independence and general quality of life of visually impaired individuals.[2]

This report will go over these developments in greater detail and describe the development of a software solution that makes great use of physical Braille displays to improve the workflow and comfort of visually impaired people using personal computers.

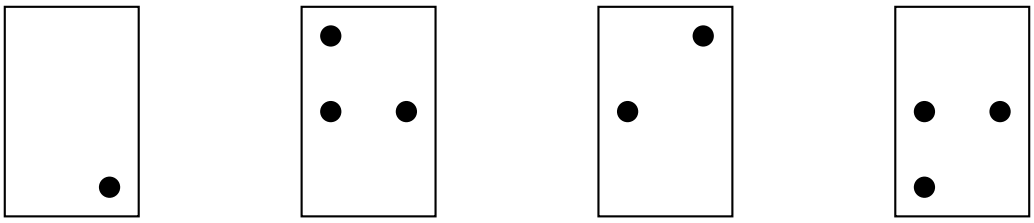
1.1 Consequences of visual impairment

It is reported by The International Agency for the Prevention of Blindness that there are 43 million people living with blindness and 295 million people living with moderate-to-severe visual impairment on a global scale.[3] The effects of vision loss can be debilitating, making simple tasks such as walking, reading, and learning difficult, or even impossible in severe cases.[4] Children with visual impairments can experience developmental delays in motor, language, cognitive and emotional skills, which can severely harm their ability to learn at school and contribute to a general lower quality of life.[4] Vision impairments also constitute a massive global economic cost, estimated by the World Health Organization to be around 411 billion US Dollars annually, primarily stemming from lost productivity.[4]
Many reports from international organizations will focus on the importance of preventing visual impairments in the first place, which is logical, since most visual impairments are treatable but often go untreated in developing countries.[4] However, less emphasis is placed on aiding individuals with untreatable visual impairments, either due to poverty or unpreventable illnesses and injuries. For these individuals, significant hurdles must be overcome to live independently, including learning how to navigate and read. The latter is in many cases impossible, if the level of impairment is too severe. If reading print by standard means is impossible, the writing system known as *Braille*, which will be explained briefly in the next section, can be a viable alternative.

1.2 Braille

Braille is a tactile writing system used by people with visual impairments to "read" information. It was developed by Louis Braille in 19th century France, after he was involved in an accident in his father's harness-making shop that left him blind.[5] Braille was originally developed as a one-to-one transliteration of the French alphabet, but through time it has been modified for many other languages, and sometimes spawned several versions within those languages.[5] Text is represented by a list of cells, each consisting of a 2x3 matrix of dots, which are either raised or lay flat in different configurations. The reader simply feels these dots to interpret shapes, symbols, words, and numbers.
With $2 \times 3 = 6$ dots that are either raised or laid flat, there are a total of $2^6 = 64$ unique braille cell configurations, which calls for special techniques to represent certain characters, given the large number of symbols, letters, and other special characters that can be present in modern documents. Depending on the system, these cells can represent single characters, groups of letters, contextual indicators, and even entire words in some cases.[6]

In the example seen in figure 1.1, the capital letter prefix " ⠠ " is used to indicate that the next alphabetic braille character is capitalized. Similarly, the capital word indicator, which is represented by two symbols ⠠⠠ is used to indicate that the next word consists of capital letters. In these ways, and many others, modern braille versions deviate significantly from standard written text.



(a) Capital letter indicator (b) "h" (c) "i" (d) "!"

Figure 1.1: "Hi!" represented in Unified English Braille

You will often see these braille dots on medicine labels, stop buttons on buses, elevators, and ATMs.

Relevant braille terms that will be used throughout the report are listed and explained below. The terms and explanations are directly quoted from *The Rules of Unified English Braille, 2nd edition*, Section 2.1[7].

- **Braille cell**
  The physical area which is occupied by a braille character
- **Braille character**
  Any one of the 64 distinct patterns of six dots, including the space, which can be expressed in braille
- **Braille sign**
  One or more consecutive braille characters comprising a unit, consisting of a root on its own or a root preceded by one or more prefixes (also referred to as braille symbol)
- **Contracted**
  Transcribed using contractions (also referred to as grade 2 braille)
- **Contraction**
  A braille sign which represents a word or a group of letters
- **Grade 1 braille**
  Used interchangeably with uncontracted
- **Grade 2 braille**
  Used interchangeably with contracted
- **Indicator**
  A braille sign that does not directly represent a print symbol but that indicates how subsequent braille sign(s) are to be interpreted
- **Mode**
  A condition initiated by an indicator and describing the effect of the indicator on subsequent braille signs
- **Terminator**
  A braille sign which marks the end of a mode

Although the original rendition of Braille was a one-to-one transliteration of an alphabet, modern versions have since evolved to use a variety of indicator symbols and contractions to increase reading speeds and save space for more Braille cells.[6] Currently, most English-speaking individuals know the Braille language known as *Grade 2 Braille*, a more compact version of *Grade 1* Braille. Unfortunately, Grade 2 Braille has a number of shortcomings that make it less ideal for use in academic and professional settings. To remedy these downsides, a new universal English standard, known as *Unified English Braille* was designed and proposed by the *International Council on English Braille*.[7] A later section will go over the aforementioned Braille systems and weigh the different benefits and downsides to determine the ideal system for our software solution.

2 Problem Delineation

2.1 The importance of Braille literacy

According to a report by the National Federation of the Blind, Braille literacy rates have been decreasing steadily in recent times, falling below 10% for blind individuals in the USA as of 2009.[8] The same report points to several factors that contribute to this negative trend. For one, there is a shortage of teachers teaching Braille and there is no national consensus on what it means to be qualified to teach Braille.[8] The report also mentions several inaccurate stigmas around Braille, such as it being slow and inefficient to read, or how it isolates its readers from other readers who read print.[8] Many schools will also encourage children with low vision to read print instead of Braille, despite it being challenging or even impossible in some cases.[8]
The report emphasizes the importance of Braille literacy and describes it as "*the key to success*"[8] for blind individuals. This claim is supported by a survey of 500 respondents, which points to "*a correlation between the ability to read Braille and a higher educational level, a higher likelihood of employment, and a higher income*"*[8]*. The same observations can be made in regard to sighted people as well. According to a report by The United Nations Educational, Scientific and Cultural Organization, "*literacy improves lives by expanding capabilities which in turn reduces poverty, increases participation in the labour market and has positive effects on health and sustainable development.*"[9].
In the United States of America, literacy for blind students is required by federal law.[10] However, many states do not enforce this law, resulting in many schools lacking adequate educators and material to support blind students.[10]

A blog post on the website of braille materials vendor *BrailleWorks* lays out different numbers that support the importance of braille literacy or explain the falling literacy rates.[10]

- "*90% of employed people with blindness can read and write braille.*"[10]
- "*60% of students with blindness drop out of school. The unemployment rate of adults who are blind hovers around 70%.*"[10]
- "*85% of students who are blind attend public schools. The braille literacy rate of the students with visual impairments is about 10%. Each year, there are fewer teachers qualified to instruct students in braille literacy.*"[10]

There is a seemingly strong link between braille literacy and higher success in education and career, which could indicate that relying on other assistive technologies is inadequate for a formal education. These assistive technologies will be described in detail in a later section. As mentioned in Section 1.1, blindness contributes to a massive global economic strain resulting from lost productivity, numbering hundreds of billions of dollars, and negatively impacts the quality of life and well-being of people who suffer from it.

2.2 Assistive technologies

There are a variety of technologies that can help visually impaired people in their daily lives. Some of the most commonly used pieces of software are *Screen Readers*, which allow users to "read" text on a computer screen with the help of *Speech Synthesis* or *Braille Displays*.[11] They apply a wide variety of techniques to extract text and other information from the computer screen, such as by interfacing with the operating system's own accessibility API or performing Optical Character Recognition on graphical elements where text cannot be extracted by other means.[11] When used together with speech synthesis, or text-to-speech tools, users can have extracted text read aloud using the computer's speakers or through headphones if the user wishes to be more discreet.[11] An interesting phenomenon occurs when visually impaired people use speech synthesis tools regularly, where they become so accustomed to listening that they can understand read-aloud speech at high playback speeds. It may sound like complete gibberish to other people, while it

is perfectly intelligible for visually impaired people who have gradually increased the playback speeds over longer periods of time.[12]
As mentioned above, screen readers can also interface with braille displays, which typically consist of a grid of pins that can be lowered or raised by small motors to function as readable braille.[13] These displays will continually update as the user moves the cursor or uses the arrow keys to select elements.[13] Braille displays have several advantages over text-to-speech tools, such as being silent, meaning that people can use them anywhere without fear of disturbing others. They also allow users to check the spelling of words and provide much faster access information.[13] Some braille displays use an extra 2 dots at the bottom of each braille cell, which allows them to display up to 256 unique symbols.[14] Many users have found this expanded system useful in areas such as mathematics.[14] The eight dots also allow for the use of Braille Computer Notation, which is a standard specified by the Braille Authority of the United Kingdom that is designed for things such as computer code. However, there is not an accepted standard system for eight-dot braille, and some people feel that advancements in computer technology may mitigate the need for eight dot displays, since many new symbols have been introduced, that the eight dot system cannot represent easily anyway.[14] The benefits and downsides of eight dot braille systems will be discussed further in a later chapter of the report.

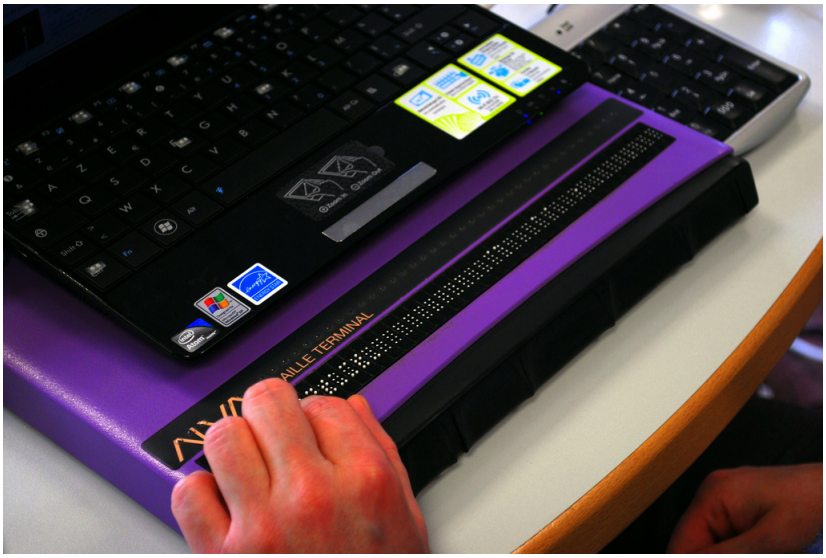


Figure 2.1: Braille display example from Wikipedia

Figure 2.1 shows a refreshable braille display with a single row of eight-dot braille cells.

One specific type of refreshable braille display is known as a *notetaker* or *brailler*, which is essentially a keyboard for typing braille on a computer.[15] Six or eight buttons that correspond to a dot in a braille cell can be pressed to type braille symbols.[15] They can often function as independent devices, or be connected to a computer to interface with its contents.[15] Braille keyboards can be a preferable alternative to standard keyboards if the user is familiar with the braille system, since standard keyboards are designed for a writing system that is quite different from braille.

While braille displays can be useful, they are unfortunately very expensive. The prices of physical braille displays typically range from $3,500 to $15,000, depending on the number of braille cells they can display, which currently makes them inaccessible to many people who simply cannot afford them.[13] Although physical braille displays are very expensive today, significant efforts are being made to lower the cost of production by researching new display technologies, which could make them available to more people.[2]

2.3 Advancements in braille display technology

As mentioned previously, most refreshable braille displays cost thousands of dollars, which severely impacts their availability for people with low incomes. This, coupled with other previously discussed points regarding lack of access to braille in schools, has resulted in many people relying on speech synthesis when interacting with technology. Thankfully, significant efforts are made to improve and reinvent the technology that powers braille displays, with the primary focus being on reducing the cost of manufacturing. One of the most affordable braille displays that is currently available is the Orbit Reader 20, which is currently priced at 700 dollars.[16] In fact, this is one of the first braille displays to cost under 1000 dollars, which is a significant leap in cost reduction for braille display technology.[17] Another significant development is the advent of market-viable multi-line braille displays, which have only been commercially available for under a decade.[17] Displays with multiple rows of braille cells are beneficial for contexts such as mathematics, music notation and spreadsheets, where the physical arrangement of the elements matters.[18] Unfortunately, despite the technological advancements, multi-line displays such as the Orbit Slate 340 are still quite expensive, selling for just under 4,000 dollars.[17]

Currently, most commercially available braille displays function by using small motors under each pin, which typically necessitates a hefty manufacturing price.[2] However, a new method of developing braille displays has the potential to change this. This new technology makes use of microfluidics to lower and raise the display's braille pins and comes with a significantly lower production cost.[2] The company behind this technology, NewHaptics, was only just recently founded in 2018, and is working to commercialize it after having received several grants from organizations such as the National Institute of Health.[2] This novel approach also helps in the production of multi-line displays, potentially making them a viable, widely available consumer product in the near future.[2]

Having looked at both contemporary, commercially available braille displays, as well as emerging braille display technologies, it is likely that a world with affordable multi-line and single line braille displays may not be too far in the future. For this reason, the primary focus of my project will be to develop software for any sort of braille display by implementing a dynamically adjustable number of lines and number of cells on each line. This should ensure that the software will be easily modifiable to work with displays of any dimension.

2.4 Problem Statement

Having discussed the problems of lacking access to sufficient braille education and the consequences of the resulting illiteracy, it becomes clear that many people could benefit from improved access to assistive braille technology. With recent revolutionary inventions in braille display technology, some refreshable braille displays have already been moved to an affordable price range, costing less than some smartphones. While many braille displays and keyboards already have on-board software to interface with screen readers and translate text to braille, with the advent of new types of displays from a variety of different manufacturers, a universal software solution that can easily be modified to work with almost any display model could prove to be a valuable tool for blind people. With all these previous points kept in mind, I have arrived at the following problem statement:

How can I design and implement a software solution to aid the visually impaired in using computers for multiple contexts with a primary focus on supporting a wide variety of Refreshable Braille Display models?

3 Design

3.1 A description of the desired product

I intend to design my software solution as a central application with a custom text-to-braille translation tool, which receives its input from a variety of sources. Depending on the type of input, the software should translate the text to the ideal braille output. For example, computer materials, such as computer code, can be translated to uncontracted braille to avoid confusion when reading the code, while text from a book should be translated to contracted braille to increase reading speeds and save space on the braille display. To show the braille output, a graphical user interface (GUI) will be implemented to simulate a refreshable braille display. This will include buttons for switching "pages" for cases where the braille does not fit on the display all at once. The GUI will also have a button for having text read aloud at a desired playback speed, in case the user is unfamiliar with a braille sign or simply wants a break from reading braille. The number of lines and number of cells on each line should both be easily configurable values, such that the program is easily adapted to a certain braille display model.

3.2 Choosing a Braille system

There are several factors to consider when choosing a braille system for our software solution. Firstly, and perhaps most importantly, the system must be familiar to a wide range of users to target the largest demographic and emphasize accessibility. The system should also have representations for a large number of symbols and special characters, such that it supports as many applications as possible. Most programming languages make use of special characters, such as the address operator "&" and the pointer operator "*" in the C programming language, which is why it is important to have Braille representations for many ASCII symbols. Since the software is intended to support a variety of different contexts, the system should also have support for areas like mathematics.

3.2.1 Grade 1

Similar to Louis Braille's original version of Braille for the French alphabet, Grade 1 English Braille is a one-to-one transliteration of the English alphabet.[6] Thus, the system has a Braille representation for each letter, with some additional punctuation symbols.

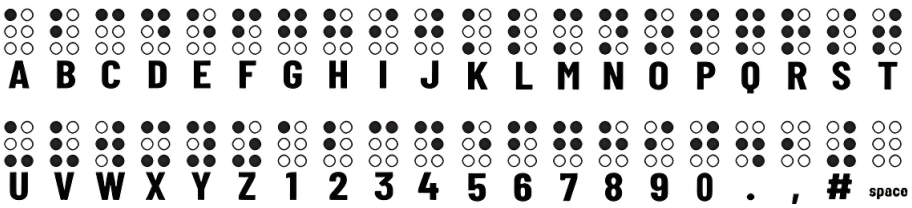


Figure 3.1: English Grade 1 Braille alphabet

This is often the first system that is taught to new Braille readers to familiarize them with a more basic version of the system. While this system is relatively simple and easy to learn, it results in Braille text taking up a lot of physical space, which is more expensive to print and takes longer to read. To amend these issues a different version, known as Grade 2 was introduced.

3.2.2 Grade 2

Grade 2 Braille makes heavy use of contractions to represent common groupings of letters or entire words to shorten the length of text. This way, some letters that would otherwise require multiple braille characters can be represented with a single character. An example of this can be seen on figure 3.1, where several contractions are used to make the braille as compact as possible.

the	b	r	ow	n	sh	o	e
⠮	⠃	⠗	⠪	⠝	⠩	⠕	⠑

Table 3.1: Grade 2 representation of "The brown shoe"

There are many other contractions that can help shorten the length of the braille, and some can only be used in specific contexts, such as at the beginning or end of a word.

	but	can	do	every	from	go	have		just
knowledge	like	more	not		people	quite	rather	so	that
us	very	it	you	as	wil				

and	ar	by was	cc con	ch child	com	dd dis	ea	ed	en enough
er	ff to	for	gg were	gh	in	ing	of	ou out	ow
sh shall	st still	the	th this	wh which	with				

Figure 3.2: Examples of contractions in Grade 2 Braille [6]

Currently, this is the most widely adopted Braille system among English speakers.[6] However, grade 2 braille has slight differences from country to country, which can introduce ambiguities when reading certain braille signs.

3.2.3 Computer Braille Code

This version of Braille was specifically designed to complement the use of computer-related materials, such as computer code and file names.[19] The version defined by the Braille Authority of North America employs the use of a standard six-dot Braille pattern to represent a variety of symbols that are commonly used in technical material.[19] Given that computer materials, such as code, will often have very irregular combinations of numbers, letters and other symbols, the system emphasizes precise representations of individual characters, without making use of contractions.[19] While this version of braille might be ideal for technical applications, it is less ideal for a software solution that is used for a wider variety of applications. For example, its lack of contractions makes it less ideal for reading standard texts, as it can significantly slow down

comprehension when readers have to read each symbol one by one.[19] It is possible to use multiple braille systems for different contexts, but this of course requires the user to learn more than one braille system, which can become a point of confusion.

3.2.4 Unified English Braille

Unified English Braille (UEB) is a relatively new system that was officially ratified by the International Council on English Braille in 2004.[7] It seeks to unify the braille symbols used by different systems in a variety of contexts, such as mathematics, programming, and standard literary texts.[7] The only context that UEB is not designed for is music notation, which is commonly done with Music braille code, also designed by Louis Braille.[20] It is designed to be as familiar as possible for people who are already familiar with other English braille systems and thus only has minor deviations from grade 2 English braille in certain areas.

Since UEB is designed for a large number of contexts, it supports a wide variety of symbols, which is ideal for a multi-purpose digital solution. It employs a number of tricks to make use of the limited configurations of a braille cell, such as sharing the same braille representation for multiple symbols, where the ambiguity is eliminated by using special indicator signs. For example, the string "abc" translates to  , whereas the string "123" translates to  . The only difference is that the translation of "123" includes the numeral indicator  as a prefix to indicate that the next braille signs are to be read as numerals instead of letters. The rules of UEB are highly complex and will be described in further detail as we develop the translation tool.

Given its similarity to the most widely used English braille system, coupled with its support for multiple contexts, I have decided to use the Unified English Braille system for the project.

4 Development

4.1 Representing braille cells digitally

Representing a single braille cell in Python is relatively straightforward. A class *Cell* is defined that simply stores which dots are raised, as well as its corresponding ASCII letter.

```
class Cell:
    def __init__(self, dots, character):
    self.dots = dots
    self.character = character
```

A *Cell* object can be simply initialized by providing a list of integers to represent the raised dots, followed by an ASCII symbol. In the code snippet below, a digital representation of the braille symbol " ⠙ " is defined.

```
cell = Cell([1,4,5], "d")
```

As a reminder, the dots in a braille cell are represented by numbers $1 - 6$, as seen in figure 4.1

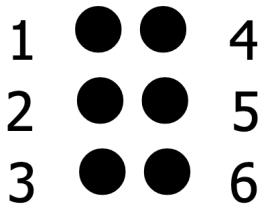


Figure 4.1: Braille dot indices
[7]

With this system in place, we can make use of hash tables to match symbols with braille dots. For example, we can define a class *Numeral* that contains a single `Cell` object. The `__init__` function will only use a single character as its parameter, which is passed to a class method called `get_dots` that returns the corresponding braille dots for the digit.

```
class Numeral:
    def __init__(self, character):
        self.cells = [Cell(self.get_dots(character), character)]

    def get_dots(self, character):
        binary_dict = {
            '1': [1],
            '2': [1, 2],
            '3': [1, 4],
            '4': [1, 4, 5],
            '5': [1, 5],
            '6': [1, 2, 4],
```

```
            '7': [1, 2, 4, 5],
            '8': [1, 2, 5],
            '9': [2, 4],
            '0': [2, 4, 5]
        }
        return binary_dict[character]
```

If an ASCII symbol is represented by multiple braille cells, we can simply map the character to a list of lists of integers and modify the `__init__` function to append a braille cell object to `self.cells` for each list of integers.

```
class GroupingPunctuation:
    def __init__(self, character):
        self.cells = []
        for dots in self.get_dots(character):
            self.cells.append(Cell(dots, character))

    def get_dots(self, character):
        binary_dict = {
            "(": [[5], [1, 2, 6]],
            ")": [[5], [3, 4, 5]],
            "[": [[4, 6], [1, 2, 6]],
            "]": [[4, 6], [3, 4, 5]],
            "{": [[4, 5, 6], [1, 2, 6]],
            "}": [[4, 5, 6], [3, 4, 5]],
            "<": [[4], [1, 2, 6]],
            ">": [[4], [3, 4, 5]],
        }
        return binary_dict[character]
```

For clarity's sake, several classes will be made to split the symbols into different groups. For example, a class `Alphabetic` is defined, which will include all letters of the English alphabet.

We will also include a number of hardcoded braille sign classes, such as a capital word indicator, which we know will always have the same two braille symbols.

```
class CapitalWordIndicator:
    def __init__(self):
        self.cells = [Cell([6], ","), Cell([6], ",")]

    def __str__(self):
        return f"Capital Word Indicator: {self.cells}"
```

The eventual output of the translator will thus be a list of these braille signs, which will be used to render the right dots on the GUI, and in theory, could be used as input for a refreshable braille display.

4.2 Braille translation

Translating print to braille can be a challenge, since some words can be composed of different combinations of contractions, single characters, and indicators. For example, translating the string "**123**" yields the numeric

indicator ⠼, followed by the braille representations of **a**, **b**, and **c**. This is a result of using the same braille representation for letters **a-j** as we do for digits **1-9** and **0**.

a	b	c	d	e	f	g	h	i	j
1	2	3	4	5	6	7	8	9	0
⠁	⠃	⠉	⠙	⠑	⠋	⠛	⠓	⠊	⠚

Table 4.1: Braille representations of letters a-j and digits 0-9

This numeric indicator is terminated by a space, but another layer of complexity is added when single words contain a combination of letters and numbers, such as in the string "**123abc**", since **123** and **abc** share the same braille representations. For these cases, the grade 1 indicator ⠰ is used to signal that the next symbols are letters instead of digits. There are many similar situations that emerge when translating print to braille, such as capital letters followed by lowercase letters, which also require two different indicators to eliminate ambiguity. Writing a translator tool entirely by hand could prove very time-consuming if all of these cases are to be kept in consideration. For this reason, I have made the choice to write a *Context-Free Grammar* to represent the structure of braille text, allowing for automatic generation of a parser that can aid in making translation more trivial.

4.2.1 Context-Free Grammars

Context-Free Grammars, or CFGs, allow us to represent languages as a set of production rules, terminals and non-terminals. CFGs are commonly notated using *Backus-Naur Form* or *Extended Backus-Naur Form*, and the former will be used to briefly showcase a simple example of a CFG.

```
<sentence> ::= <subject> + <action> + <object>
```

In the above example, a single production rule is defined for a simplified version of the English language. On the left-hand side of the production rule, we see the *nonterminal* "`<sentence>`", while on the right, we see a composition of nonterminals `<subject> + <action> + <object>`. This effectively communicates that a "sentence" nonterminal can be derived into these three other nonterminals. However, every nonterminal in a language must derive into *terminals* for the language to be valid. Let us introduce a production rule for each of the three new nonterminals to address this.

```
<subject> ::= 'John' | 'Mark' | 'Mary' | 'Beth'

<action> ::= 'eats' | 'walks' | 'plays with' | 'talks to'

<object> ::= 'the dog' | 'the cat' | 'the apple' | <subject>
```

On the left hand side of each rule, we again see the corresponding nonterminal. However, on the right hand side, we now see a variety of terminals, each enclosed with apostrophes to indicate that they are strings. Notice that the `<object>` rule can also be derived into a `<subject>`, allowing for more valid strings. With these rules introduced, every nonterminal can be derived into a terminal, resulting in a complete language. Let us now generate a string using the rules defined by our CFG. We will use a technique called *left-most derivation* for this example, where each nonterminal is derived step-by-step, starting from the left.

```
<sentence> ⇒ <subject> + <action> + <object>
           ⇒ 'John' + <action> + <object>
           ⇒ 'John' + eats the + <object>
           ⇒ 'John' + eats the + apple
```

The resulting string is "John eats the apple".
With tools such as *ANTLR*, we can use grammars written in Extended Backus-Naur Form (EBNF) to automatically generate a parser, which can help transform our input string to a form that is more useful during the translation process. The benefits of this will become more apparent as we go over the creation of the braille CFG and the output of the resulting parser.

4.2.2 A Context-Free Grammar for grade 1 Unified English Braille

We can consider braille to be a list of words separated by spaces, much like any other language. However, we will consider any substring of symbols, be it letters, numbers or special symbols, to be a word, as long as it is surrounded by spaces. Our grammar will not serve to ensure the grammatical correctness or syntactic validity of the string we wish to translate, since the input can take a number of forms, such as computer code or literary texts. The grammar will simply help us translate strings of every possible permutation to its braille equivalent. Let us express this idea using ANTLR's version of EBNF.

```
text
    :   (space* word (space+ word?)*)* EOF
    ;
```

Having looked at the previous CFG example, this should look familiar, as we are again looking at a production rule, albeit in a slightly different form. The nonterminal `text` represents the entire input string that we wish to derive from our language. On the right-hand side, we see a regular expression that may look a bit intimidating. To explain the expression in layman's terms, `text` can consist of any combination of words and spaces followed by End of File (EOF), as long as words are separated by at least one space. The grammar will also accept a string with no words or spaces at all.
Next, we define the production rules for a `word`.

```
word
    :   (sequence | single)+
    ;
```

This rule defines a word as being composed of one or more sequences or single letters, where a sequence refers to a list of characters from the same category, such as a numeral sequence or a lowercase letter sequence. The "one or more" part is denoted by the "+" operator after `sequence`, similar to the "*" operator, which means "zero or more".

```
sequence
    :   (capitals_sequence | numeral_sequence | lowercase_sequence | symbol_sequence)
    ;
```

The production rule for `sequence` can be seen above. The rule simply determines that a `sequence` can be derived into any of the four sequence types, these being a capital letter sequence, a numeral sequence, a lowercase letter sequence, and a special symbol sequence.
The terminals for our CFG will be defined by lexer rules, which are written in a fashion quite similar to parser production rules. We will define lexer rules for lowercase letters, uppercase letters and digits.

```
Lowercase
    :   [a-z]
    ;

Uppercase
    :   [A-Z]
```

```
    ;

Digit
    :   [0-9]
    ;
```

In the lexer rule for `Lowercase`, we specify that a Lowercase token can represent any letter from a to z using `[a-z]`. Similar regular expressions are used to define the other rules.

For defining special symbols, we will split them into several categories, such as `grouping_punctuation`, which will contain things like parentheses, square brackets and curly brackets. A symbol sequence can be derived into any of these production rules to match any symbol we define in our production rules.

```
punctuation
    :    ',' | '.' | '\'' | ':' | '_' | '-' | '!' | '-' |
         '?' | ';' | '...' | '/' | '\\' | '‘' | '’' | '‘' | '’' |
         '"'
    ;

grouping_punctuation
    :   '(' | ')' | '[' | ']' | '{' | '}' | '<' | '>'
    ;

op_and_comp
    :   '+' | '=' | '*' | '^'
    ;

currency_and_measurement
    :   '€' | '$' | '£'
    ;
```

These rules cover some of the most commonly used symbols, but they are nowhere near exhaustive.

We can use ANTLR's preview tool to see the output of the generated parser for a given string. Let us try this with the string "`abc123 *!?`". This will produce the following output:

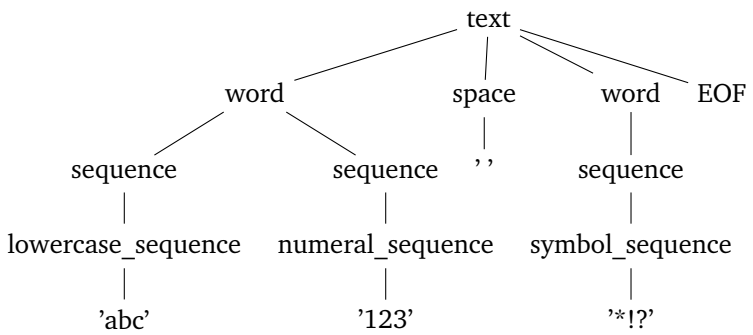


Figure 4.2: Parse tree for the string "**abc123 *!?**"

This data structure is called a *parse tree*. It consists of a root node, in this case, a "text" node, with four child nodes. At the bottom of the tree, we see three leaf nodes, each containing a substring of the input string. Note that in reality, every character in a sequence is represented by their own separate nodes, but for the sake of visual clarity, they will be represented as a single node. We can write an algorithm that "traverses"

the tree to generate the desired braille symbols. In its current state, the parse tree does not provide us with many benefits in the translation process compared to simply iterating through the leaf nodes one by one. To amend this, some changes will be made to the CFG.

```
capitals_sequence
    :   uppercase uppercase+ capitals_terminator?
    ;

capitals_terminator
    :   lowercase_sequence | symbols_sequence
    ;
```

A new nonterminal `capitals_terminator` is introduced, which can potentially follow a capitals sequence in cases where it precedes one or more lowercase letters or a symbols sequence. This is added to follow the standard specified by Section 8.6.1 in The Rules of Unified English Braille: "*The capitals terminator is placed after the final capitalised letter either within or following the symbols-sequence.*"[7].

Thus, the string 'ABCa' should translate to the braille seen on figure 4.3

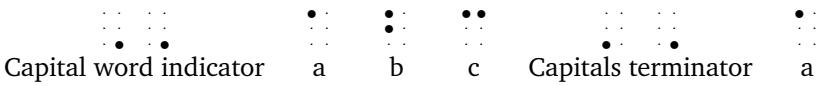


Figure 4.3

The parse tree for this string would look as follows:

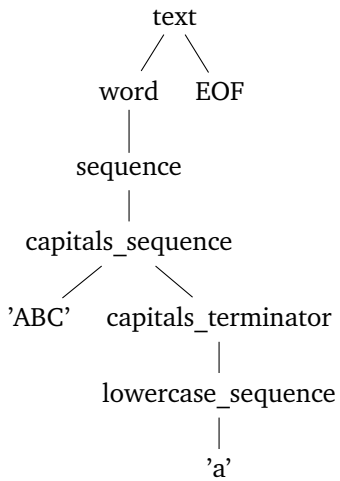


Figure 4.4: Parse tree for the string "**ABCa**"

Now, whenever a `capitals_terminator` node is encountered while traversing the parse tree, we will know that a capitals terminator braille sign should be appended to the output. Similar special cases can now be added to the CFG relatively quickly, such that it is in accordance with the rules of UEB, for example, when a numeral sequence is followed by one or more lowercase letters, where a grade 1 terminator is needed to terminate the numeral mode.

With these rules in place, a basic CFG for uncontracted Unified English Braille has been implemented, which we will use for some translations of technical material, such as computer code, again, according to the standard specified in the rules of UEB, Section 11.10.2, Technical Material[7].

4.2.3 A Context-Free Grammar for grade 2 Unified English Braille

While grade 1 UEB is useful in certain contexts, such as when interacting with technical materials, it is less useful when reading more typical text, such as a book or a news article. For these cases, we will write a CFG for *grade 2* UEB, which makes use of contractions to speed up reading times and save physical space for more braille cells. The rules of grade 2 UEB are complex and require much more advanced production rules to express with a CFG. We will start off by implementing Strong Contractions, since they have relatively simple rules compared to other contractions and shortforms. These words and their corresponding braille contraction are listed below.[7]

and 

for 

of 

the 

with 

Section 10.3.1 of The Rules of Unified English Braille specifies: "*Use the strong contraction wherever the letters it represents occur unless other rules limit its use.*"[7] This means in most cases, we can simply replace any occurrence of those words, even within another word, with a single braille symbol. We will write a lexer rule that tells the lexer to tokenize any occurrence of any of these words to a `STRONG_CONTRACTION` token.

```
STRONG_CONTRACTION
:
    'and' | 'for' | 'of' | 'the' | 'with'
;
```

However, this will only tokenize any occurrence where the word is written in lowercase letters. A capital letter indicator can be used in front of a strong contraction to indicate that the first letter of the contraction is a capital letter and likewise, a capital word indicator will make the entire contraction have capital letters. For these reasons, we will have three different tokens for each type of strong contraction, such that they can only be used in the right contexts, and to make the translation process easier.

```
STRONG_CONTRACTION_L
    :
        'and' | 'for' | 'of' | 'the' | 'with'
    ;

STRONG_CONTRACTION_C
    :
        'AND' | 'FOR' | 'OF' | 'THE' | 'WITH'
    ;

STRONG_CONTRACTION_F
```

```
    :
        'And' | 'For' | 'Of' | 'The' | 'With'
    ;
```

Now, we can introduce these lexer rules to some of the parser rules, such as `lowercase_sequence`, where lowercase strong contractions can now be part of the sequence.

```
lowercase_sequence
    :   (strong_contraction_l | lowercase)+
    ;
```

With these additions, the string "`before`" will generate the parse tree seen in Figure 4.5

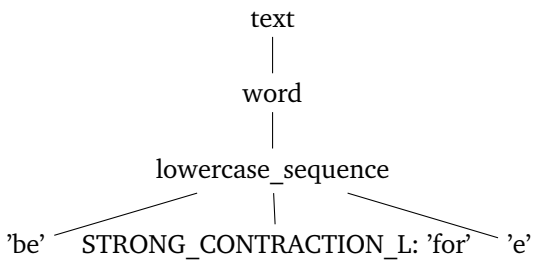


Figure 4.5: Simplified parse tree for the string "before"

Next, we can implement another type of contraction, known as an Alphabetic Wordsign. The idea is that the braille representation for every letter except "i" and "a" has a separate meaning if it "stands alone". In this way, a single letter standing alone can represent a common word, such as "can" or "like". The rules for when a word or a sequence is standing alone are slightly complex and will be explained more in-depth later. For now, we will treat every word that is preceded or followed by a letter or digit as not standing alone.

If we simply defined a lexer rule for each word that can be represented with an alphabetic wordsign as we did before with strong contractions, the lexer would generate tokens in incorrect contexts, such as with the input string "butter", which would lex "but" to an alphabetic wordsign token, since "but" is one of the words that can be represented by a single letter. The rest of the letters would be lexed to lowercase letter tokens. Thus, we need a way to avoid tokenization for these contexts. We do this by using the `@members` action block at the top of the lexer rules to add custom Python code to the automatically generated lexer.

```
@members {
def not_alnum(self, direction):
    target_char = chr(self._input.LA(direction)) if self._input.LA(direction) >= 0 else ''
    return not target_char.isalnum()
}
```

We use this together with *semantic predicates* within the lexer rules to ensure that words are only tokenized to alphabetic wordsign tokens in the right context. Semantic predicates are defined using `{}?` and evaluate the code inside the curly brackets to a boolean value, where any evaluation to *False* will prevent the tokenization of the word. Thus, both `not_alnum(-1)` and `not_alnum(1)` must be true for the tokenization process to be successful. Otherwise, the letters within the words will be tokenized as individual lowercase letters.

```
ALPHABETIC_WORDSIGN_L
    :   {self.not_alnum(-1)}?
```

```
        ('but' | 'can' | 'do' | 'every' | 'from' | 'go' |
         'have' | 'just' | 'knowledge' | 'like' | 'more' |
         'not' | 'people' | 'quite' | 'rather' | 'so' |
         'that' | 'us' | 'very' | 'will' | 'it' | 'you' | 'as')
        {self.not_alnum(1)}?
    ;
```

When `self._input.LA(-1)` is called in the first semantic predicate, it will perform a look-behind operation and check the character positioned one letter behind the start of the word that is being tokenized. We convert the retrieved integer value to its corresponding character and ensure that it is neither a letter nor a digit using `return not target_char.isalnum()`. Similarly, when `self._input.LA(1)` is called in the second semantic predicate, it will perform a lookahead operation to check the character positioned one letter in front of the final character in the word that is being tokenized. If other checks need to be made for other words, we can simply append a new function in the `@members` action block that implements the specific context we want to check for. With these rules implemented, the string "but" will generate a single alphabetic wordsign token, while "butter" will generate a lowercase letter token for each letter.

Another type of contraction that is very similar to strong contractions is the Strong Groupsign, which shortens many common letter combinations to a single sign. Like strong contractions, they are used whenever the letters they represent are used within a word or anywhere else, unless rules specify otherwise.[7]

ch	gh	sh	th	wh	ed	er	ou	ow	st	ing	ar
⠡	⠣	⠩	⠹	⠱	⠫	⠻	⠳	⠪	⠌	⠬	⠜

Table 4.2: Strong Groupsigns and their print equivalents

For example, "stowing" translates to "⠌⠕⠺⠬", which is significantly shorter than "⠎⠞⠕⠺⠊⠝⠛", its uncontracted equivalent.

We will now look closer at the concept of "standing alone". Section 2.6.1 of The Rules of UEB specifies that "*A letter or letters-sequence is considered to be "standing alone" if it is preceded and followed by a space, a hyphen or a dash. The dash may be of any length, i.e. the dash or the long dash.*"[7].

We define a production rule `standing_alone_connector` that encapsulates these symbols.

```
standing_alone_connector
    :   HYPHEN | DASH | LONG_DASH
    ;
```

Then this production rule can be used as a prefix and suffix in a `standing_alone` rule.

```
standing_alone
    :   standing_alone_connector? standing_alone_part
    (standing_alone_connector? | (standing_alone_connector standing_alone_part)*)
    ;
```

Using this rule, we can treat the contents of `standing_alone_part` differently, such as using a grade 1 indicator for single letters, so that they are not mistaken for alphabetic wordsigns. Multiple standing alone parts can also be chained together using the second option within the parenthesis.

4.2.4 Generating braille cells

The process of traversing and processing the parse tree was briefly discussed in the previous section, but this section will describe the process in more rigorous detail. In addition to automatically generating a lexer and parser for our CFG of choice, ANTLR will also generate a `Visitor` class, with functions for "visiting" each node in the tree. If we invoke the function for visiting the root node, it will recursively visit each child node, starting from the left. If a visited child node has child nodes of its own, the leftmost node will again be visited first. This continues until the entire tree has been traversed. This algorithm is known as a *Depth-First Search* or DFS.
A visualization of the order in which nodes are visited can be seen in Figure 4.6

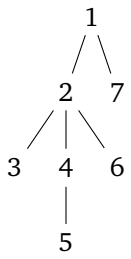


Figure 4.6: Tree showing the order in which nodes are visited in a DFS

Let us write now write a class that inherits from the automatically generated `uncontracted_brailleVisitor`. We initialise it with an empty list, `symbols`, where we will be storing instances of the classes we showed an example of in section 4.1. After traversing the tree, we iterate through the list of symbols and extract all braille cells, which are added to a list and returned from the function `generate_cells()`.

```
class CellGenerator(uncontracted_brailleVisitor):
    def __init__(self):
        self.signs = []

    def generate_cells(self, text):
        self.visitText(text)

        cells = []

        for symbol in self.signs:
            for cell in symbol.cells:
                cells.append(cell)
        return cells
```

Below, we see an example of a visitor function, which by default will simply invoke the visitor functions for its children. However, by specifying alternative behavior, we can override its default functionality. In this case, we still visit every child node in the numeral sequence, which in this case would be every digit, but we also append a numeral indicator to `signs`, to indicate that the next braille signs are numerals.

```
def visitNumeral_sequence(self, ctx: uncontracted_brailleParser.Numeral_sequenceContext):
        self.signs.append(NumeralIndicator())
        return self.visitChildren(ctx)
```

Every visitor function that we leave as is will simply visit its children. Thus, we only need to overwrite a

visitor function if adding a braille sign is required, such as when visiting a wordsign. Let us visualize the process with an example, where we wish to translate the string "From the get-go" using the contracted braille cell translator. This will produce the following parse tree.

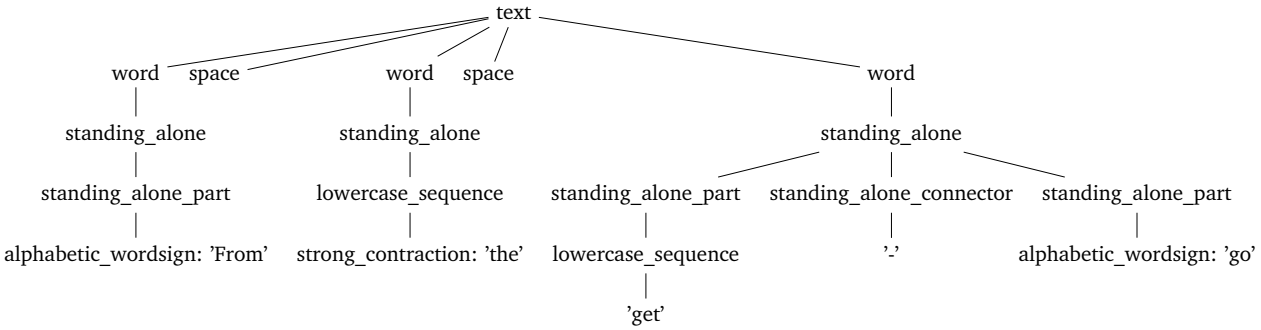


Figure 4.7: Parse tree for the string "From the get-go"

When we run the contracted braille cell generator on this parse tree, we will again start at `text`, where its leftmost child is visited first. Only the leaf node `alphabetic_wordsign: 'From'` of this branch has an overridden visitor function. We perform a simple check to see which type of token we are dealing with. In this case, 'From' is an `ALPHABETIC_WORDSIGN_F` token, where only the first letter of the word is capitalized. Thus, we will append a `CapitalFirstLetter` braille sign to `self.signs`. If the token were instead `ALPHABETIC_WORDSIGN_C`, the entire word would be capitalized, in which case we would append a `CapitalWordIndicator` to the list. The final token type simply has all lowercase letters, in which case we do not need to append any indicators. Before returning from the visitor function, we append an `AlphabeticWordsign` braille sign to the list, which contains the text from the token.

```
def visitAlphabetic_wordsign(self, ctx: contracted_braille_parser.Alphabetic_wordsignContext):
    if ctx.ALPHABETIC_WORDSIGN_F():
        self.signs.append(CapitalFirstLetter())
    if ctx.ALPHABETIC_WORDSIGN_C():
        self.signs.append(CapitalWordIndicator())
    self.signs.append(AlphabeticWordSign(ctx.getChild(0).getText()))
    return
```

In the second branch of the tree, the same action is performed on the `strong_contraction` leaf node, which has a nearly identical visitor function, except a `StrongContraction` braille sign is appended to the list.

The third branch has three leaf nodes of its own: a lowercase sequence, a hyphen and an alphabetic wordsign. When visiting the lowercase sequence, the visitor function will perform a check on its string to ensure that the letters do not match a shortform. For example, the braille representation of the letters "ab", ⠠⠠, is identical to the shortform of "about", ⠠. Thus, we need to append a Grade 1 Symbol Indicator to avoid ambiguity.

```
def visitLowercase_sequence(self, ctx:contracted_braille_parser.Lowercase_sequenceContext):
    string = ""
    for letter in ctx.children:
        string += letter.getText()

    shortform_letters = ['ab', 'ac', 'af', 'afw', 'alm', 'alt', 'alw']
```

```
7
8       if string in shortform_letters:
9           self.signs.append(Grade1SymbolIndicator())
10
11      return self.visitChildren(ctx)
```

For the hyphen, we simply append the braille sign that represents it.

```
1   def visitStanding_alone_connector(self,
2   ctx:contracted_braille_parser.Standing_alone_connectorContext):
3       self.signs.append(Punctuation(ctx.getChild(0).getText()))
4       return
```

For the alphabetic wordsign, we simply repeat the process from the very first leaf node in the tree. If we render the generated braille cells, we get the result seen in Figure 4.8

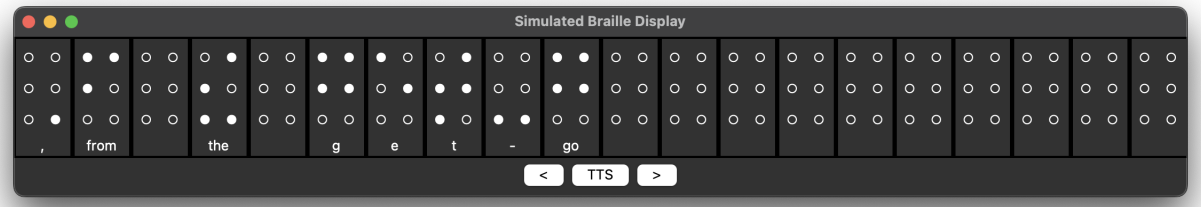


Figure 4.8: Contracted braille representation of "From the get-go"

4.3 Simulating the output of a refreshable braille display

We will make use of a package called *Tkinter*, which allows us to use a user interface toolkit called TK. This will be used to render the generated braille cells for our input string, as well as to simulate the interactive elements of a braille display, such as buttons to switch between pages when the full result does not fit on the simulated display. We will build this GUI step-by-step, starting by defining the main window. We define a class `MainWindow`, that implements `tk.Tk`. The class will also contain a list of pages each containing braille cells, a list that keeps track of which cells are currently being displayed and an integer value that keeps track of the current page number.

```
1   class MainWindow(tk.Tk):
2           def __init__(self):
3           super().__init__()
4           self.title("Braille Display Simulation")
5           self.geometry(f"{WINDOW_WIDTH}x{WINDOW_HEIGHT}")
6           self.cell_pages = []
7           self.cells_on_page = []
8           self.current_page = 0
```

We will now implement a class method that renders a target input string, which is invoked with an empty string on startup to initialise the empty cells.

```
def render_braille_cells(self, data: dict) -> None:
    string = get_target_string(data)
    type = data['type']
```

First, the exact string wish to translate is retrieved using `get_target_string()`. The object `json_data` contains all the relevant information we need to construct the intended string. If `data_type == 'vscode'`, we will know that the source of the data object was Visual Studio Code. Data objects with this type will contain two extra fields `line_number` and `line_text`, where the former holds the line number of the line of code stored in the latter. Both of these are combined into a single string, which is returned from the function. If the type of the data object is simply `text`, the raw text will be returned from the function. When any other data sources are added in the future, they will simply be handled in this function to retrieve a target string for braille translation.

```
def get_target_string(json_data: dict) -> str:
    data_type = json_data['type']

    if data_type == 'vscode':
        line_number = json_data['line_number']
        line_text = json_data['line_text']
        target_string = f"{line_number}: {line_text}"
        return target_string
    elif data_type == 'text':
        return json_data['content']
```

Next, we convert the target string to a list of braille cells using `get_cells()`. We use the field `is_contracted` from the data object to determine which braille translator to use. Data sources, such as data from a code editor, will likely have uncontracted braille as its desired output. We perform a simple check on this value and choose the corresponding translator. The string is lexed to a list of tokens, which is converted to a parse tree. We then use the `cell_generator` to traverse the parse tree and generate and return the desired braille cells.

```
def get_cells(data: dict) -> list:
    string = get_target_string(data)
    is_contracted = data['is_contracted']

    input_stream = InputStream(string)

    if is_contracted:
        cell_generator = ContractedCellGenerator()
        lexer = contracted_braille_lexer(input_stream)
        token_stream = CommonTokenStream(lexer)
        parser = contracted_braille_parser(token_stream)
    else:
        cell_generator = UncontractedCellGenerator()
        lexer = uncontracted_brailleLexer(input_stream)
        token_stream = CommonTokenStream(lexer)
        parser = uncontracted_brailleParser(token_stream)

    ast = parser.text()

    cells = cell_generator.generate_cells(ast)
    return cells
```

We will now define a function that converts the cell objects to Tkinter widgets. A widget is a graphical element that can be rendered on a window, and we will make use of these to render the braille cells. We will simply use the list of raised dots and the cell's corresponding character when an instance of `BrailleCellWidget` is initialized.

```
def convert_to_widgets(cells: list) -> list:
    widgets = []
    for cell in cells:
        widgets.append(BrailleCellWidget(None, cell.dots, cell.character))
    return widgets
```

The class `BrailleCellWidget` will be initialized with these values and run a method `create_labels()` to add the dots as visual elements to the cell.

```
class BrailleCellWidget(tk.Frame):
    def __init__(self, master, dots, character):
        super().__init__(master, borderwidth=2, relief="solid")
        self.dot_labels = None
        self.dots = dots
        self.character = character
        self.create_labels()
```

This function will loop through all six dot indices and place a lowered dot in each spot. It will also create a text label underneath the dots that renders its corresponding character.

```
def create_labels(self) -> None:
    # Create labels for each dot
    self.dot_labels = []
    for i in range(6):
        label = tk.Label(self, text="o", font=("Arial", 20))
        label.grid(row=i % 3, column=i // 3)
        self.dot_labels.append(label)

    # Create label for character
    character_label = tk.Label(self, text=self.character, width=5)
    character_label.grid(row=3, column=0, columnspan=2)  # Span two columns
```

A second class method `update_display()` will be used to lower or raise the dots correctly, according to the values stored in `self.dots`.

```
    def update_display(self) -> None:
        # Update the display based on the state of the dots
        dot_symbols = ["o", "*"]  # Not raised and raised dot symbols
        for i in range(6):
            if i + 1 in self.dots:
                self.dot_labels[i]["text"] = dot_symbols[1]
            else:
                self.dot_labels[i]["text"] = dot_symbols[0]
```

Going back to the `render_braille_cells` method, we will determine the max number of braille cells that can be rendered on the display at once. We simply multiply the number of rows by the number columns with `max_cells = MAX_ROWS * MAX_COLUMNS`. In this case, the number of rows will be `1` and the number of columns will be `20`, which is the same layout as on the Orbit Reader 20. These two numbers are simply hardcoded constants within the main code file, and they should be changed according to the target display.

We will use the maximum number of braille cells on a page to populate the `self.pages` list. This will result in a list of a list of braille cell widgets. We use `self.cells_on_page` to specify which of the pages we wish to render and call `update_display()` to place the braille widgets on the main window.

```
# Create a list of pages, each page containing max_cells number of cells
self.cell_pages = [cell_widgets[i:i + max_cells] for i in range(0, len(cell_widgets), max_cells)]

# Set the current page of cells
self.cells_on_page = self.cell_pages[self.current_page]

# Place the Braille cells on the window
self.update_display()
```

The first step of the `update_display` method is to remove the "old" widgets, which will later be replaced with the new widgets we wish to render. Since there are no widgets currently placed on the window, this will do nothing.

```
def update_display(self) -> None:
    # Remove all cells from the grid
    for widget in self.grid_slaves():
        if isinstance(widget, BrailleCellWidget):
            widget.grid_forget()
```

Next, we will loop through the list of cells to render on the page and use their `grid` property to arrange them correctly. We will use the index of each cell to determine its position on the grid. For example, the first cell has an index of 0. Two equations are used to calculate the correct row and column number for the cell.

$$\text{row\_index} = \lfloor \frac{\text{index}}{\text{MAX\_COLUMNS}} \rfloor$$
$$\text{column\_index} = \text{index} \bmod \text{MAX\_COLUMNS}$$

Given `cell_index = 24, MAX_COLUMNS = 20, MAX_ROWS = 2`

```
row_index = 24 // 20 -> 1
column_index = 24 % 20 -> 4
```

Thus, the cell will be placed in the fifth column of the second row (since their coordinates start at 0).

```
# Place the cells on the grid
    for cell_index, cell in enumerate(self.cells_on_page):
```

```
        row_index = cell_index // MAX_COLUMNS
        column_index = cell_index % MAX_COLUMNS
        cell.grid(row=row_index, column=column_index)
        cell.update_display()  # Update the display of the Braille cell widget
```

As a final step, we need to ensure that the rest of the space on the page is populated with empty cell widgets if there are fewer cells on the page than the maximum number of cells that can be displayed.

```
max_cells = MAX_ROWS * MAX_COLUMNS
if len(self.cells_on_page) < max_cells:
    for i in range(len(self.cells_on_page), max_cells):
        row_index = i // MAX_COLUMNS
        column_index = i % MAX_COLUMNS
        empty_cell = BrailleCellWidget(self, [False] * 6, " ")
        empty_cell.grid(row=row_index, column=column_index)
```

With these components implemented, we can run the window using `window.mainloop()`. This results in the following GUI being rendered.

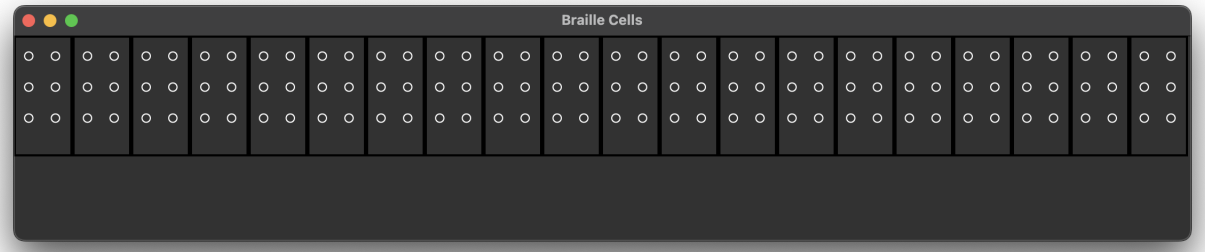


Figure 4.9: Simulated output of a braille display with 1 line and 20 cells per line

4.4 Inter-Process Communication

Now that we have implemented a GUI to simulate the braille display and two different braille translators, we can set up a method to receive data from a variety of sources. This could be from a code editor, a Microsoft Word document, a web browser or any piece of software we can write plugins for. This will be achieved using a TCP socket connection, which will continually listen for data while the program is running. We will define a class `SocketHandler` with a reference to the main window, which will be responsible for handling any data that is received via a socket connection.

```
class SocketHandler:
    def __init__(self, main_window):
        self.sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        self.sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        self.sock.bind(('localhost', PORT))
        self.sock.listen(1)

        self.main_window = main_window
```

The class will contain a method `listen_for_data()`, which will continually run on a separate thread and decode any incoming data. If a valid JSON object is received and decoded, we will call the `render_braille_cells` method using the json object as an argument.

```
    def listen_for_data(self):
        try:
            conn, addr = self.sock.accept()
            while True:
                data = conn.recv(1024)
                if not data:
                    break
                json_data = json.loads(data.decode('utf-8'))
                if json_data:
                    self.main_window.render_braille_cells(json_data)
        except Exception as e:
            print(f"Error: {e}")
            print("Attempting to reconnect...")
        finally:
            if conn:
                conn.close()
            self.sock.close()
```

With this approach, any application that can communicate over a TCP connection will able to interface with our software. Unfortunately, web browsers, such as Firefox do not have support for TCP sockets in their extensions. Firefox instead has support for websockets. This necessitates the use of a separate server to handle any data we wish to receive from a web browser. The process of implementing this separate server will be described in a later section.

4.5 Visual Studio Code integration

Visual Studio Code or VSCode is a common choice for a code editor among programmers. Although relatively bare-bones by itself, it boasts a large collection of user-made extensions, which implement or modify features in a variety of ways. It is relatively trivial to implement a custom extension due to its extensive documentation and API functions. We will implement a simple extension that fetches the line of code at which the cursor is currently positioned, appends the line number behind the line of code, and transmits the data to the braille display software.

We will define a function *activate*, that runs when the extension is loaded. The function will invoke another function *connectToServer()*, which will attempt to establish a connection to the braille display software. If the attempt at connecting is unsuccessful, the function will be invoked again after five seconds and likewise if it disconnects after a successful connection.

```
export function activate(context: vscode.ExtensionContext) {
    connectToServer();
    ...
```

After successfully establishing a connection to the braille display software, we add a listener that is invoked whenever the user moves their cursor in the code editor window. We then access the *editor* object, through which we can access the position and text of the currently selected line of code. We simply combine these into a JSON object, which is transmitted over the TCP/IP socket.

```
    ...
disposable = vscode.window.onDidChangeTextEditorSelection((event) => {
    const editor = vscode.window.activeTextEditor;
    if (editor) {
```

```
        const position = editor.selection.active;
        const currentLine = editor.document.lineAt(position.line);
        const currentLineText = currentLine.text;

        if (socket && !socket.connecting) {
            let data = JSON.stringify({
                line_number: position.line + 1,
                cursor_position: position.character,
                line_text: currentLineText
                is_contracted: false
            });
            socket.write(data);
            console.log('Sent:', data);
        }
    }
});
```

While only the relevant aspects of the extension have been outlined in this section, the full code numbered below one hundred lines code, highlighting the ease at which we can interface with the braille display software. Many additions can be made to the extension, such as ways to transmit file names, terminal output, in-line suggestions and error messages.

4.6 Translating text from images

To aid in reading text from images, we will implement a small secondary server that runs in tandem with our primary braille display software server. This secondary server will be responsible for performing optical character recognition (OCR) on images containing text. For transmitting images to the server, we will implement a Firefox plugin, that adds a new option to the right click menu for images.

```
connect();

browser.menus.onClicked.addListener(async function (info, tab) {
  if (info.menuItemId == "braille-ocr") {
    console.log("Sending image to OCR server");
    const response = await fetch(info.srcUrl);
    const blob = await response.blob();
    ...
```

In a manner very similar to the VSCode extension, we will start off by invoking a *connect()* function, that also attempts to establish a websocket connection to our image OCR server every five seconds until it is successful. We then add a listener that runs when we press the button in the right click menu with the id `braille-ocr`, which is defined at the beginning of the extension. We then fetch the image data based on its URL. Next, we convert the image to a base64 encoded string and store it in a JSON object with a field called 'type', which is included in case we wish to extend the functionality beyond images in the future. The base64 string is stored in the 'content' field and the JSON object is sent on the websocket to the OCR server.

```
    ...
    // Convert the image to a Base64 string
    const reader = new FileReader();
    reader.onloadend = function() {
      const base64data = reader.result;

      // Create a JSON object with the image data and an indicator
      const data = JSON.stringify({
        type: 'image',
        content: base64data
      });

      // Send the image over the socket
      socket.send(data);
    }
    reader.readAsDataURL(blob);
  }
});
```

Finally, let us go over the process of performing OCR on the specified image on the OCR server. We will initially attempt to connect to the braille display software server, again attempting to connect every five seconds until it is successful.

```
def connect_socket():
    global sock
    while True:
        try:
            # Close the socket if it's already open
            if sock:
                sock.close()
                sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            sock.connect(server_address)
            # If the connection is successful, break the loop
            break
        except Exception as e:
            print(f"Error: {e}. Trying to reconnect in 5 seconds.")
            time.sleep(5)
```

Next, we will listen for data on the websocket connection between the OCR server and the Firefox plugin, which we have connected to earlier in the program. If we receive a JSON object with `type == image`, we will invoke the `ocr_image()` function.

```
async def echo(websocket, path):
    async for message in websocket:
        data = json.loads(message)
        if data['type'] == 'image':
            await ocr_image(data['content'])
```

The relevant image data is decoded and stored as an `Image` object. To perform OCR on the image, we will be using the open source library *Pytesseract*, which allows us to use a pre-trained machine learning model to recognize text on an image. We simply run the `image_to_string` library function and transmit the resulting text to the braille display server. The string is stored in a JSON object with type `text`, such that the text will be translated to contracted braille.

```
async def ocr_image(data_url):
    # Split the '' part from the actual image data
    mime, base64_string = data_url.split(',')

    base64_bytes = base64.b64decode(base64_string)
    image_bytes = io.BytesIO(base64_bytes)
    image = Image.open(image_bytes)
    # Perform OCR on the image
    text = pytesseract.image_to_string(image)

    # Create a json object to send to the server
    json_data = json.dumps({'type': 'text', 'content': text})
    while True:
        try:
            # Send the data to the server
            sock.sendall(json_data.encode('utf-8'))
            # If the data is sent successfully, break the loop
            break
        except Exception as e:
            print(f"Error: {e}. Trying to reconnect in 5 seconds.")
            connect_socket()
```

4.7 Text-to-speech support

As a supplementary tool for our braille display software, text-to-speech (or TTS) support will be included. This can be useful in cases where a user does not recognize one or more braille signs, but knows the corresponding symbol by name. The process is relatively straight-forward for standard text, such as books, where the raw text is simply used as input for the TTS software or library to generate an audio file of spoken English. Natural sounding TTS software is an understandably common preference, but for the purposes of this project, I will be using the *gTTS* python package, which interfaces with the Google Translate text-to-speech API to generate basic TTS audio files for free. This API can easily be swapped out for another more advanced and paid API in the future if the need arises. For playing the audio files, I will be using the *Pygame* package, which includes an easy-to-use sound engine.

We define a function `read_text`, which takes the input text as an argument. The function initialises a gTTS object, generates an audio file from the input text, plays it using pygame and finally deletes the audio file, which was temporarily stored in a `temp` folder. This entire process is run on a separate thread when the function is invoked to ensure that the rest of the program does not halt when the TTS system is used.

```
def read_text(text):
    # Create a gTTS object for the text
    tts = gTTS(text, lang='en')
    # Save the audio as a temporary file
    tts.save("temp/tts.mp3")
    try:
        # Initialize pygame mixer
        pygame.mixer.init()
        # Load the audio file
        pygame.mixer.music.load("temp/tts.mp3")
        # Play the audio
        pygame.mixer.music.play()
        # Wait for playback to finish
        while pygame.mixer.music.get_busy():
            pygame.time.Clock().tick(10)
    finally:
        # Remove the temporary file
        if os.path.exists("temp/tts.mp3"):
            os.remove("temp/tts.mp3")
```

Although this works well for normal text, it is not adequate if the user wishes to have technical material, such as computer code, read aloud. When a string, such as "`printf("Hello World!\n");`" is fed to the gTTS API, it will often omit special characters such as parentheses, curly brackets and quotation marks. Since these special symbols are often crucial in writing and understanding computer code, some way must be found to force the engine to read every single symbol aloud. To address this, the `read_text` function will receive an extra boolean argument to specify whether the input string should be read aloud as standard text or technical material. Depending on this boolean argument, we will simply modify the input string itself to replace each occurrence of a special symbol with its phonetic spelling. For example, "**Hi!**" becomes "**Hi exclamation mark**". Additionally, every occurrence of the letter "**a**" standing alone will be converted to "**ay**", such that the phonetic spelling of the letter will be pronounced, instead of the word. We define a function containing a large hash map, where a special symbol is used to retrieve the corresponding phonetic spelling of the symbol as a string.

```
def get_string_from_char(char):
    binary_dict = {
```

```
        " ": ".",
        "{": "left curly bracket",
        "}": "right curly bracket",
        "(": "left parenthesis",
        ")": "right parenthesis",
        ...
        }
    if char in binary_dict:
        return binary_dict[char]
    else:
        return False
```

Then we simply run this function for every character in the input string to obtain the desired output.

5 Discussion

5.1 Future improvements

Handling high data transmit rates
It is important for refreshable braille displays to work in real-time, with as little delay as possible between receiving an input and showing it on the display. Unfortunately, my program is unable to handle rapid transmittals of data. For example, if you move the cursor around rapidly while using the VSCode extension, the display will eventually be stuck on the same input string. It is still possible to switch pages and use the text-to-speech button, but no new data can be received. This can happen both temporarily and permanently, requiring a restart of the software. A potential fix for this could be to improve the netcode in the backend, such that it has more error handling and fail-safes that ensure that it will attempt to reestablish its connection instead of permanently breaking.

Extending the context-free grammars
While the CFGs already cover many commonly used symbols and combinations of symbols, there are some that are not fully covered by the grammars. The contracted braille grammar in particular has several areas with home for improvement. For one, not every type of contraction has been implemented, since the rules for their usage can be highly complex. Some irregular combinations of characters will also not be recognized by the contracted grammar, particularly when standing alone sequences are present, where any special symbol following a standing alone connector will result in an error. Implementing the contracted grammar proved to be highly complex and unfortunately left many unaddressed cases with erroneous outputs.

Interfacing with accessibility APIs
Modern operating systems, such as Windows and MacOS, come with native accessibility APIs, which can help with extracting text from user interface elements and navigating around the computer. Future iterations of my program could benefit greatly from these features and could improve the general user experience.

Adding adjustable playback speeds for synthesized audio
The text-to-speech function currently only supports a single playback speed of synthesized speech. While text-to-speech is not a primary focus for the software, having the option to adjust playback speeds could improve the user's productivity and general user experience. If some part of the text causes confusion for the user, being able to slow down the playback speed could help with comprehension of words, symbols or the general layout of text elements. If the user has adapted to higher playback speeds, which some blind people do, being able to speed up the playback could cause a significant boost to productivity.

Adding extensions for more contexts
The final product only has external plugins for receiving data from two sources, those being from lines of code in Visual Studio Code and from images in Firefox. Adding more plugins for other pieces of software would be a beneficial addition to the project. For mathematics, a plugin could be written to extract equations from MatLab, which is a piece of software for writing and calculating mathematical problems. Another context could be musical notation, which could be done with MuseScore, a piece of software for writing and listening to sheet music. Both of these could make use of the program's support for multiple lines of braille cells, such as by displaying the bars from sheet music stacked on top of each other, or by showing equations in a more accurate manner than what is possible with a single line.

5.2 Reflecting on the design and development process

The process of designing and developing my software solution was unfortunately not without its hurdles. For one, working alone required a significantly higher time investment to achieve goals that could attained faster with more members on the development team. During previous semesters, we have spent considerable amounts of time getting used to delegating work and working on specific components of our software solutions. This allowed us to familiarize ourselves more with fewer parts of the project, leading to an over-

all higher degree of software quality. Single-handedly designing and developing everything from scratch is quite unfamiliar to me, which also resulted in major amounts of lost productivity. I unfortunately ended up spending a lot of time simply thinking about what to do next with the project, and I often lacked ideas for features and improvements.

Major losses in productivity were caused by frequent errors while writing the context-free grammar for contracted braille. ANTLR4, the tool that I used to automatically generate the lexer and parser for the translators had several unintended behaviours while writing the lexer rules. This resulted in incorrect tokenizations of some strings, which led to a large number of errors during the development process. While ANTLR4 has a decent amount of documentation for its Java version, there was less to be found regarding its Python equivalent. Even with consultation from experienced ANTLR users, the problems were left unsolved for a long time. Eventually, through rigorous trial and error, I was able to solve the tokenization problem using semantic predicates, which set me back on track with the contracted braille CFG.

As a consequence of working alone, I had no obligations toward any other individuals during the development process. Some of these obligations, such as having daily meetings where team members expect to see each other physically and work together can be massive contributors toward productivity and general motivation on the team. Without these obligations, maintaining a regular work schedule proved to be incredibly difficult. With better planning and a more structured work schedule, significantly more work could have been put into designing, developing and improving the project.

A major mistake that I made during the project was to never schedule any meetings with my supervisor, which primarily stemmed from a lack of motivation. Having someone to share ideas with and receive regular feedback from could have been a major help during the development of the display program and while writing the report.

6 | Conclusion

After a discussing the different areas with room for improvement, both in regard to the state of the final product and the development process itself, it becomes clear that the project could have greatly benefited from a more structured approach. More correspondence with my supervisor could have been a great help while writing the report and developing the project itself. Significant setbacks during the development of the CFG for contracted UEB took days of troubleshooting to address, leading to a partial implementation of its many rules. Adding functionality for interfacing with operating system accessibility APIs could have expanded its use cases and made good use of its Inter-Process Communication support. Despite several shortcomings during its development, the final product includes a variety of tools and functionality, such as two custom translation tools for translating print to contracted and uncontracted Unified English Braille. Its heavy application of IPC allows it to receive data from a wide variety of sources via TCP sockets and websockets, while treating the incoming data in a fitting manner, depending on its type.

In context of the problem statement:

How can I design and implement a software solution to aid the visually impaired in using computers for multiple contexts with a primary focus on supporting a wide variety of Refreshable Braille Display models?

I would mark the solution proposal as being mostly realized. There is support for multiple contexts in the form of computer code through the Visual Studio Code plugin and support for standard text through the Firefox plugin. Implementing support for other contexts and applications should remain relatively trivial, given the ease at which external extensions that communicate with the braille display software could be implemented. There are many aspects of the project that could receive improvements in the form of optimizations, error handling, expansion of grammars and extended support for contexts.

Bibliography

1. *Braille displays: A blind spot of the mainstream tech industry*, https://arstechnica.com/gadgets/2016/03/alternative-cheaper-braille-displays/, Accessed: 2024-04-25.
2. *NewHaptics works to Commercialize Multiline Braille and Graphical Tactile Display, Nicknamed the "Holy Braille" Project*, https://innovationpartnerships.umich.edu/stories/newhaptics-works-to-commercialize-multiline-braille-and-graphical-tactile-display-nicknamed-the-holy-braille-project/, Accessed: 2024-04-25.
3. *Vision Atlas*, https://www.iapb.org/learn/vision-atlas/causes-of-vision-loss/, Accessed: 2024-05-12.
4. *Blindness and vision impairment*, https://www.who.int/news-room/fact-sheets/detail/blindness-and-visual-impairment, Accessed: 2024-05-14.
5. *Louis Braille and the Braille System*, https://www.duxburysystems.com/braille.asp, Accessed: 2024-04-25.
6. *What Is Braille?*, https://brailleworks.com/braille-resources/what-is-braille/, Accessed: 2024-04-25.
7. *Rules of Unified English Braille*, https://iceb.org/ueb.html, Accessed: 2024-04-25.
8. *The Braille Literacy Crisis in America*, https://nfb.org/images/nfb/documents/pdf/braille_literacy_report_web.pdf, Accessed: 2024-04-25.
9. *What you need to know about literacy*, https://www.unesco.org/en/literacy/need-know, Accessed: 2024-05-07.
10. *Why is Braille Literacy So Critical?*, https://brailleworks.com/braille-literacy-vital-academic-improvement-employment/, Accessed: 2024-05-18.
11. *Screen readers*, https://www.afb.org/blindness-and-low-vision/using-technology/assistive-technology-products/screen-readers, Accessed: 2024-05-18.
12. *How Do Blind People Use Technology? | Convos With Julia*, https://www.youtube.com/watch?v=5s7d5ozHRFo, Accessed: 2024-05-21.
13. *Refreshable Braille Displays*, https://www.afb.org/node/16207/refreshable-braille-displays, Accessed: 2024-05-21.
14. *Eight-dot Braille*, https://www.brailleauthority.org/eight-dot-braille, Accessed: 2024-05-24.
15. *What is a Braille Keyboard?*, https://allyant.com/what-is-a-braille-keyboard-types/, Accessed: 2024-05-24.
16. *Orbit Reader 20*, http://www.orbitresearch.com/product/orbit-reader-20/, Accessed: 2024-05-28.
17. *Spotlighting Breakthroughs in Multi-line Braille Displays*, https://www.afb.org/blog/entry/spotlighting-multi-line-displays, Accessed: 2024-05-28.
18. *Orbit Slate 340 – Multi-line Refreshable Braille Display*, https://www.orbitresearch.com/product/orbit-slate-340-multi-line-refreshable-braille-display/, Accessed: 2024-05-28.
19. *COMPUTER BRAILLE CODE 2000*, https://www.brailleauthority.org/cbc/cbc2000.pdf, Accessed: 2024-05-03.
20. *MUSIC BRAILLE CODE, 2015*, https://www.brailleauthority.org/music/Music_Braille_Code_2015.pdf, Accessed: 2024-05-06.